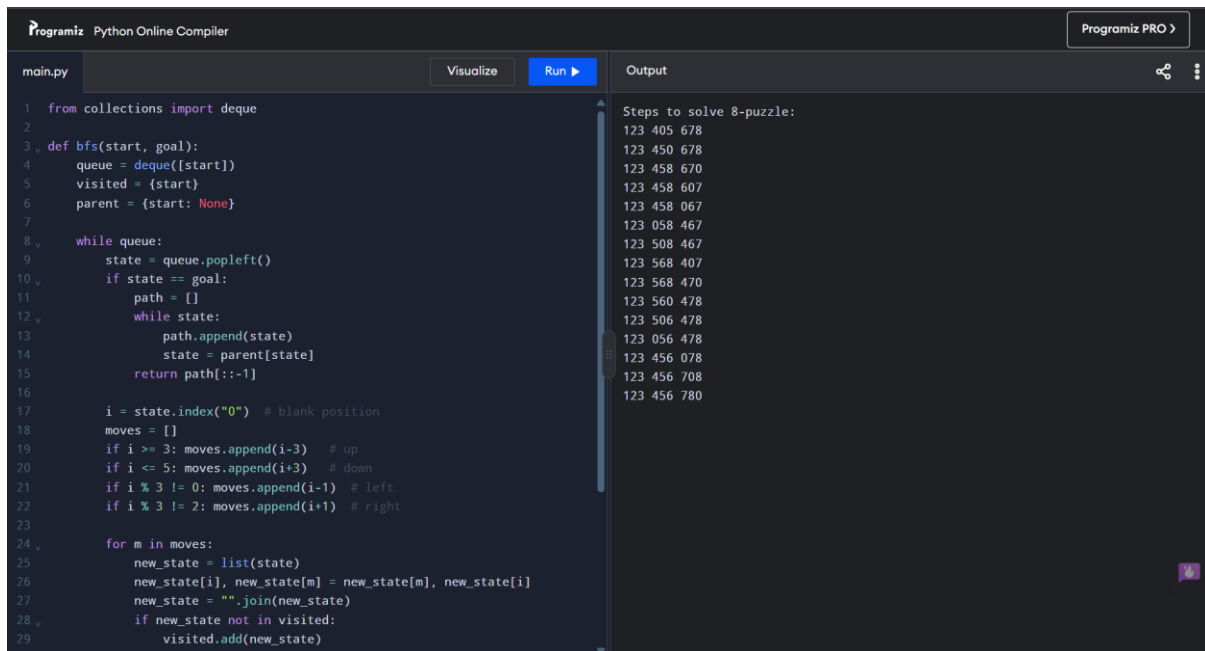


COURSE CODE & NAME : CSA1712

ARTIFICIAL INTELLIGENCE.

Lab exp 01:



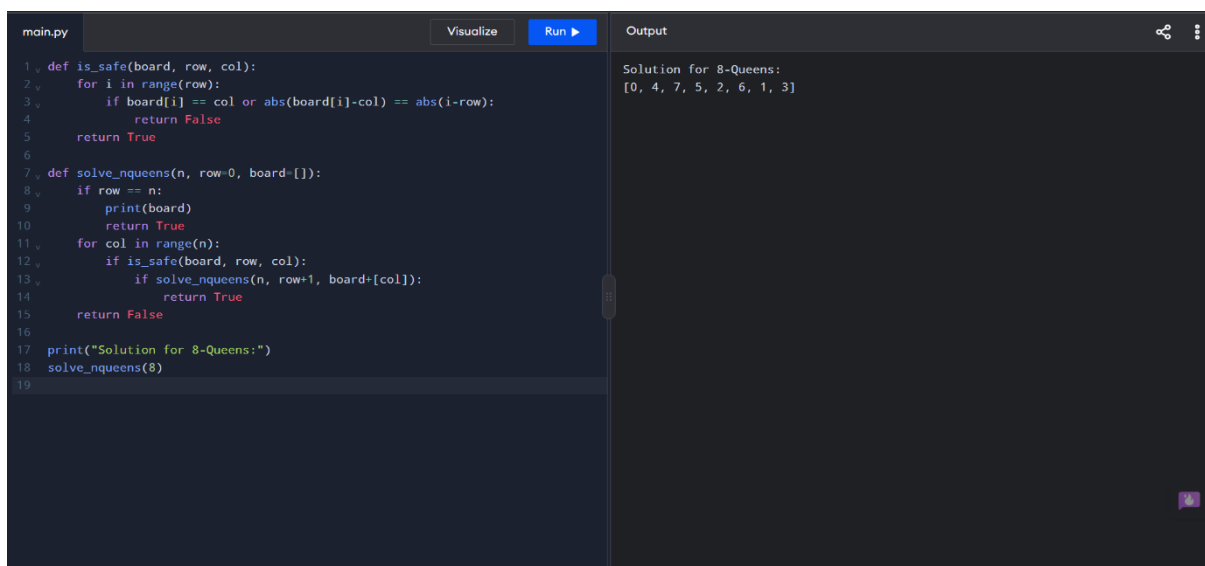
The screenshot shows a Python Online Compiler interface with a file named 'main.py'. The code implements a Breadth-First Search (BFS) algorithm to solve an 8-puzzle. The puzzle state is represented as a string of 8 characters, where '0' is the blank space. The algorithm starts from an initial state and explores all possible moves (up, down, left, right) until it reaches the goal state. The output shows the steps to solve the puzzle, listing the sequence of states from the start to the goal.

```
1 from collections import deque
2
3 def bfs(start, goal):
4     queue = deque([start])
5     visited = {start}
6     parent = {start: None}
7
8     while queue:
9         state = queue.popleft()
10        if state == goal:
11            path = []
12            while state:
13                path.append(state)
14                state = parent[state]
15            return path[::-1]
16
17        i = state.index("0") # blank position
18        moves = []
19        if i >= 3: moves.append(i-3) # up
20        if i <= 5: moves.append(i+3) # down
21        if i % 3 != 0: moves.append(i-1) # left
22        if i % 3 != 2: moves.append(i+1) # right
23
24        for m in moves:
25            new_state = list(state)
26            new_state[i], new_state[m] = new_state[m], new_state[i]
27            new_state = "".join(new_state)
28            if new_state not in visited:
29                visited.add(new_state)
```

Output:

```
Steps to solve 8-puzzle:
123 405 678
123 450 678
123 458 670
123 458 607
123 458 067
123 058 467
123 508 467
123 568 407
123 568 470
123 560 478
123 506 478
123 056 478
123 456 078
123 456 708
123 456 780
```

Lab exp 02:



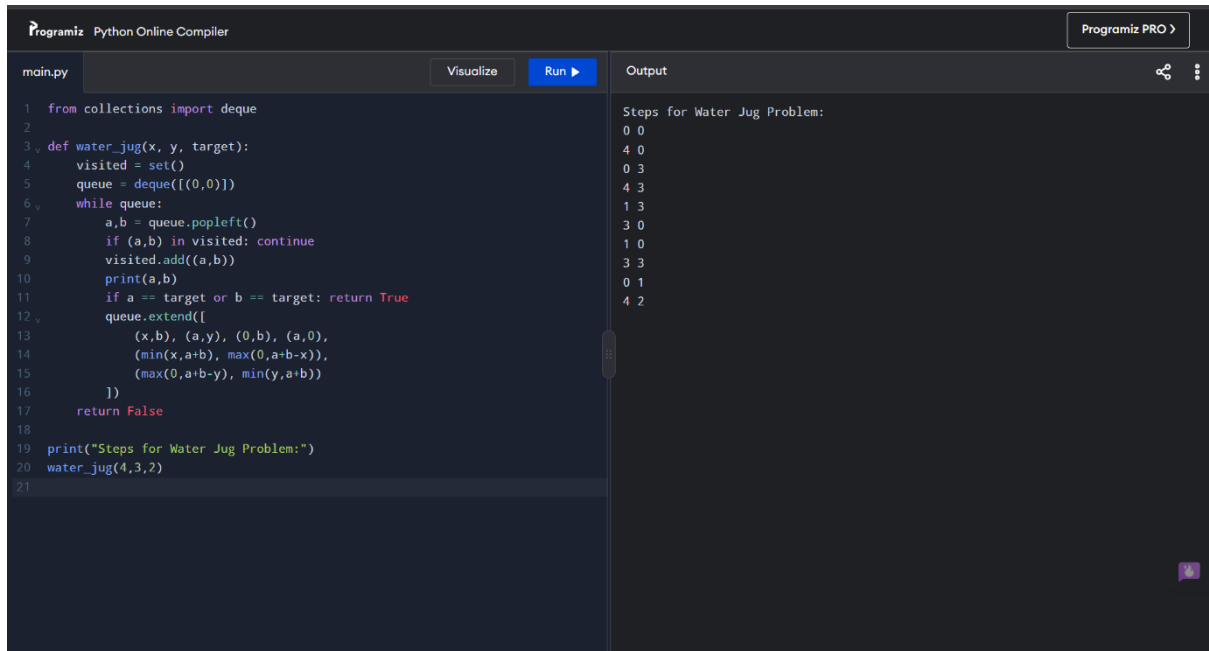
The screenshot shows a Python Online Compiler interface with a file named 'main.py'. The code implements a backtracking algorithm to solve the 8-Queens problem. It checks if a queen can be placed at a given position without conflicting with other queens already on the board. If a solution is found, it prints the row indices of the queens for each column.

```
1 def is_safe(board, row, col):
2     for i in range(row):
3         if board[i] == col or abs(board[i]-col) == abs(i-row):
4             return False
5     return True
6
7 def solve_nqueens(n, row=0, board=[]):
8     if row == n:
9         print(board)
10        return True
11    for col in range(n):
12        if is_safe(board, row, col):
13            if solve_nqueens(n, row+1, board+[col]):
14                return True
15    return False
16
17 print("Solution for 8-Queens:")
18 solve_nqueens(8)
```

Output:

```
Solution for 8-Queens:
[0, 4, 7, 5, 2, 6, 1, 3]
```

Lab exp 03:



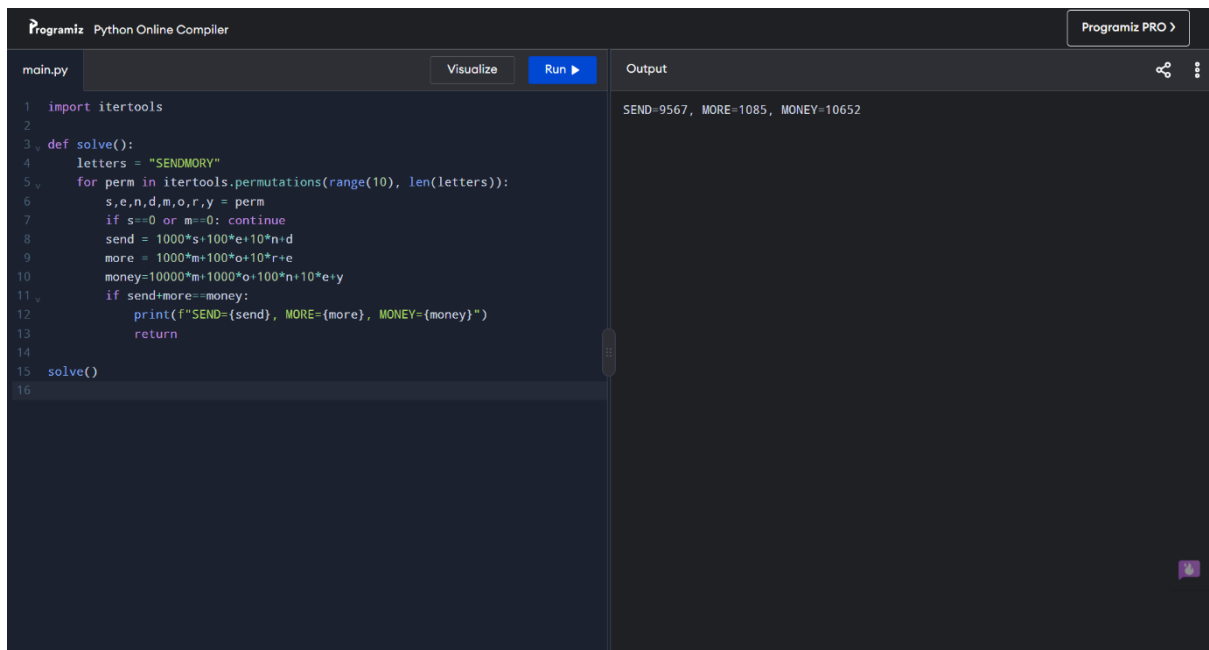
The screenshot shows the Programiz Python Online Compiler interface. The code in the editor is as follows:

```
1 from collections import deque
2
3 def water_jug(x, y, target):
4     visited = set()
5     queue = deque([(0,0)])
6     while queue:
7         a,b = queue.popleft()
8         if (a,b) in visited: continue
9         visited.add((a,b))
10        print(a,b)
11        if a == target or b == target: return True
12        queue.extend([
13            (x,b), (a,y), (0,b), (a,0),
14            (min(x,a+b), max(0,a+b-x)),
15            (max(0,a+b-y), min(y,a+b))
16        ])
17    return False
18
19 print("Steps for Water Jug Problem:")
20 water_jug(4,3,2)
21
```

The output on the right is:

```
Steps for Water Jug Problem:
0 0
4 0
0 3
4 3
1 3
3 0
1 0
3 3
0 1
4 2
```

Lab exp 04:



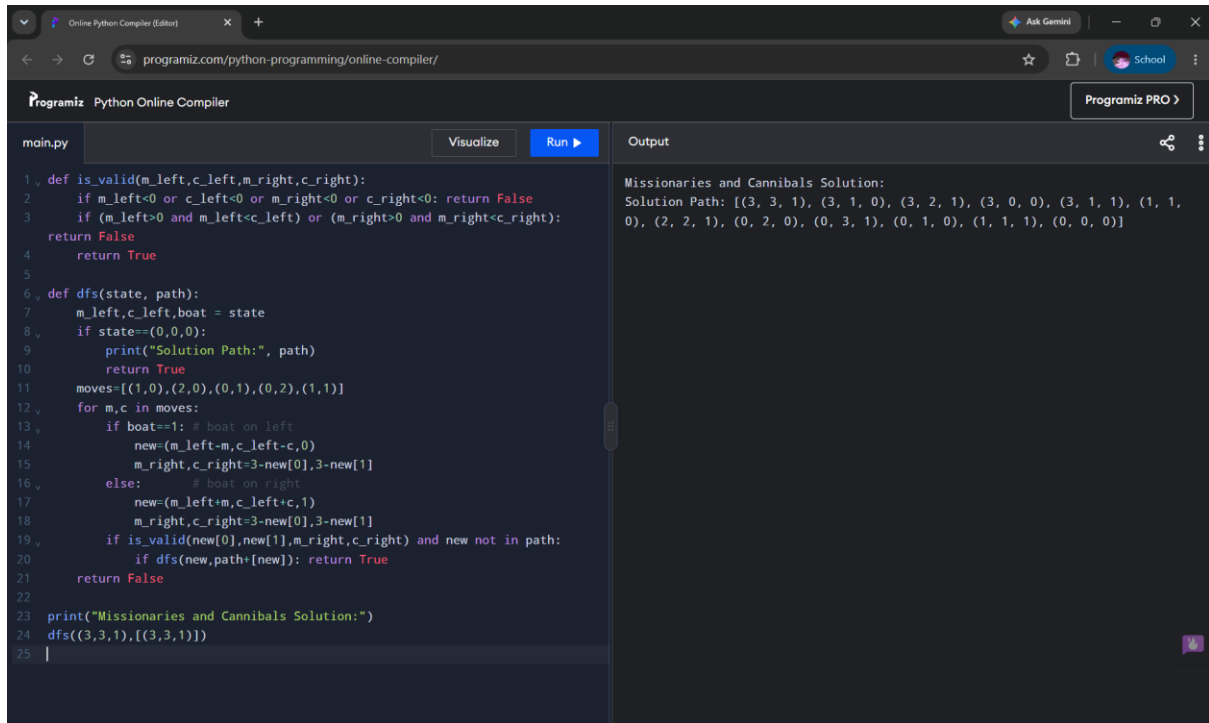
The screenshot shows the Programiz Python Online Compiler interface. The code in the editor is as follows:

```
1 import itertools
2
3 def solve():
4     letters = "SENDMORY"
5     for perm in itertools.permutations(range(10), len(letters)):
6         s,e,n,d,m,o,r,y = perm
7         if s==0 or m==0: continue
8         send = 1000*s+100*e+10*n+d
9         more = 1000*m+100*o+10*r+e
10        money=10000*m+1000*o+100*n+10*e+y
11        if send+more==money:
12            print(f"SEND={send}, MORE={more}, MONEY={money}")
13            return
14
15 solve()
16
```

The output on the right is:

```
SEND=9567, MORE=1085, MONEY=10652
```

Lab exp 05:



The screenshot shows a web browser window with the URL `programiz.com/python-programming/online-compiler/`. The page title is "Programiz Python Online Compiler". The interface includes a "main.py" tab, "Visualize" and "Run" buttons, and an "Output" panel.

The code in the editor is as follows:

```
1 def is_valid(m_left,c_left,m_right,c_right):
2     if m_left<0 or c_left<0 or m_right<0 or c_right<0: return False
3     if (m_left>0 and m_left<c_left) or (m_right>0 and m_right<c_right):
4         return False
5     return True
6
7 def dfs(state, path):
8     m_left,c_left,boat = state
9     if state==(0,0,0):
10        print("Solution Path:", path)
11        return True
12    moves=[(1,0),(2,0),(0,1),(0,2),(1,1)]
13    for m,c in moves:
14        if boat==1: # boat on left
15            new=(m_left-m,c_left-c,0)
16            m_right,c_right=3-new[0],3-new[1]
17        else: # boat on right
18            new=(m_left+m,c_left+c,1)
19            m_right,c_right=3-new[0],3-new[1]
20        if is_valid(new[0],new[1],m_right,c_right) and new not in path:
21            if dfs(new,path+[new]): return True
22    return False
23
24 print("Missionaries and Cannibals Solution:")
25 dfs((3,3,1),[(3,3,1)])
```

The output panel displays the following text:

```
Missionaries and Cannibals Solution:
Solution Path: [(3, 3, 1), (3, 1, 0), (3, 2, 1), (3, 0, 0), (3, 1, 1), (1, 1,
0), (2, 2, 1), (0, 2, 0), (0, 3, 1), (0, 1, 0), (1, 1, 1), (0, 0, 0)]
```